V-Escape

Project Team

Abdullah Shahid Butt	21I-0721
Rizwan Salim	21I-0574
Salman Jan	21I-2574

Session 2021-2025

Supervised by

Mr. Shams Farooq

Co-Supervised by

Mr. Muhammad Wasif Ali Wasif

Department of Computer Science

National University of Computer and Emerging Sciences
Islamabad, Pakistan

June, 2025

Contents

List of Figures

List of Tables

Chapter 1

Introduction

V-Escape is a horror game which has procedural map generation and a story to go along with the levels to keep the players of the game invested in the game. This document is meant to be read by all the stake holders of this project (FYP pannel members, Supervisors of the project, Players, Designers, Programmers, Artists) for them to be able to understand what is the skeleton of our project.

1.1 Problem Statement

Games featuring dungeons and escape rooms face several notable challenges. One major issue is that these games can quickly become repetitive. In video games, players often become bored when they grow too familiar with fixed maps and puzzles that don't change much, leading to a decrease in excitement and replay value which is evident in games like 'Granny'[1], 'Dark Occult'[2] and 'Escape Room'[3].

Additionally, difficulty scaling can be an issue[4]. Some games do not adapt well to different difficulties like 'Granny'[1], whether they are physical or digital games. This can limit the game's appeal to a broader audience.

Moreover, many dungeon and escape room games like 'Dark Occult'[2] feature maps that are visually similar, offering limited variety. While having a consistent theme is important, this can make the game visually monotonous over time. The lack of visual diversity further contributes to the game feeling stale and less engaging.

1.2 Scope

At the start of the game the player will be able to change some simple game settings from the options menu, start the game or exit it. When the player selects the Play button, a map is procedurally generated of the initial level and multiple map components are added to it by further procedural generation.

In this process first a number of areas or zones are defined as locked and unlocked and the whole system of the level completion requirement is set in place and at this stage by spawning in the required objects to solve the puzzles so that the stage is solvable by the player and these objects along with other environment objects are also generated procedurally to make sure they don't look out of place and do not disturb the immersive experience of the player.

The Player is allowed to move around the space and collect any collectable objects and use them to solve the puzzles. The puzzles include labyrinths, weight puzzles, pattern puzzles, memory puzzles, and some simple "find key to the next door" puzzles.

Once the player has completed the puzzles and escaped the initial setting (which will be a building/house/open area) they proceed to the next part of the game which are outdoors, houses and once again that map is also procedurally generated along with the items in it. This continues on in a loop with the player entering a new environment with new puzzles after each completion, depending on the game's difficulty the maps can be just a few or just enough to give the player a challenge, each stage getting harder than the last one.

Levels will also have an agent (villain/ghost/monster) that will chase the player and once a player is caught he will lose lives and re-spawn in one of the already visited rooms. The agent will not be too complex. It would just respond to certain things like seeing the player, hearing noise, or just wander around hoping to catch the player.

Once all the players lose all their lives a game over screen appears and the game ends with a prompt to restart from the beginning.

1.3 Modules

We have 4 different modules in our project, which are as follows:

1.3.1 Diverse Map Variations

This module uses procedural generation to create varied map layouts, themes, and challenges, ensuring fresh gameplay. It prevents repetition by adjusting map complexity, room types, and interactive elements.

1. Multiple Maps with completely different setting to provide visual diversity.
2. All maps will still have a sense of horror to them to keep the player immersed in the experience.

1.3.2 Procedural Map Generation

This module ensures that procedurally generated maps maintain logical coherence while placing environment-suitable objects. It guarantees that essential items for progression are appropriately positioned, preventing them from looking out of place.

1. The map will have randomness in its layout every time it is generated to make sure players don't really get used to it over time.
2. Maps will be generated on a certain seed number allowing players to recreate the maps if the same seed number is provided.

1.3.3 Puzzle Generation

This module strategically places challenges, such as locked rooms and keys, within the procedurally generated map. This design encourages players to explore different areas and revisit established rooms to collect essential items for progressing to new sections.

1. Puzzles will be places in sequence of adding locked and unlocked rooms.
2. Every locked room will be unlock-able from the items available in currently unlocked rooms.
3. Unlocking items needed will be scattered sequentially as the game decides the play flow the player is supposed to take.

1.3.4 Narrative Creation

A compelling narrative enhances player engagement by providing context and motivation for their actions. As levels progress, multiple narratives will link to the maps, keeping the story believable and adding depth to gameplay.

1. The Narrative will tell the players the story of the main character of the game.
2. It will keep the players invested in the game.

1.4 User Classes and Characteristics

User Class	Description
Casual Players	Players who enjoy occasional gaming sessions and prefer easy-to-understand mechanics. They are interested in light entertainment without investing too much time or effort into mastering complex gameplay.
Hardcore Players	Gamers who enjoy deep, challenging experiences and spend significant time mastering games. They often look for difficult puzzles, complex challenges, and rewarding gameplay mechanics.
Puzzle Enthusiasts	Players who are specifically drawn to games with complex puzzle mechanics. They enjoy testing their problem-solving skills and look for puzzles that vary in difficulty and type.
Horror Enthusiasts	These players are drawn to games with a strong horror element, seeking tension, fear, and atmospheric settings. The horror aspect of "V-Escape" with the chasing agent and unsettling environments will be key for this group.
Speed Runners	Players who aim to complete games as quickly as possible, focusing on efficiency and minimizing time spent in each level. They often look for optimal routes and strategies to bypass obstacles.
Completionists	These players aim to explore and achieve everything within the game. They look for all hidden items, solve every puzzle, and experience all possible game content.

Table 1.1: User Classes and Characteristics

Chapter 2

Project Requirements

2.1 Use-case/Event Response Table/Storyboarding

Following is the use-case diagram of our game project.

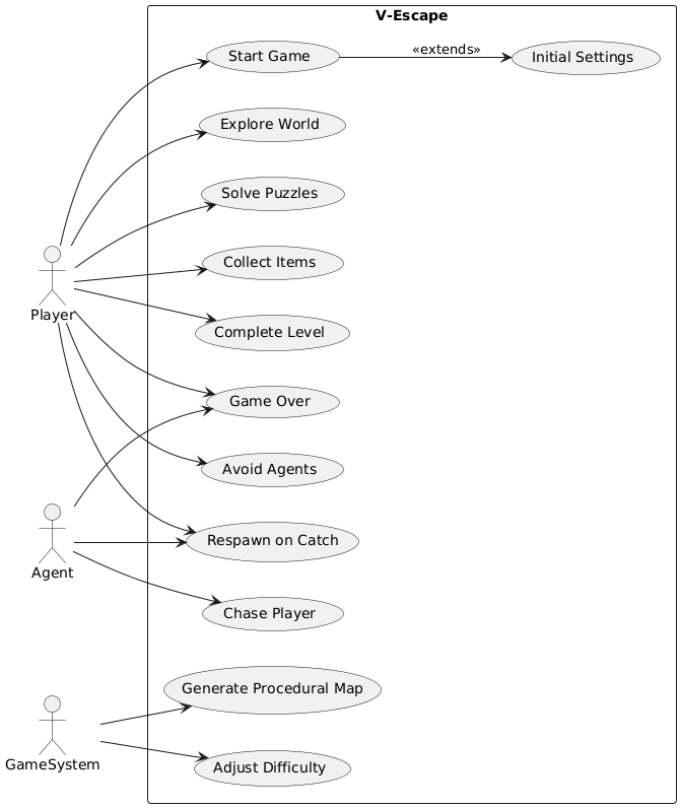

Figure 2.1: Use-case Diagram

The following are the detailed uses-cases:

Use Case Name	USC1: Procedural Map Generation
Level	User Goal
Primary Actor	Game System.
Stakeholders	Player
Preconditions	• The game must have started. • The system must be capable of generating maps based on predefined difficulty levels and conditions.
Main Success Scenario	• The player selects the "Start Game" option. • The game system uses procedural generation algorithms to create a new map. • The map is generated with a unique layout, puzzles, and item locations. • The map adjusts difficulty settings based on the player's profile (e.g., skill level, previous performance). • The generated map is loaded into the game for the player to explore.
Postconditions	• The player enters a procedurally generated environment. • The game world is unique and reflects the appropriate difficulty for the player's current level.

Table 2.1: Detailed Use-case 1

Use Case Name	USC2: Solve Puzzle
Level	User Goal
Primary Actor	Player.
Stakeholders	Game System
Preconditions	• The player has entered a puzzle section of the map. • The puzzle is available for interaction.
Main Success Scenario	• The player approaches and interacts with the puzzle object (e.g., door lock, cipher, lever). • The game system presents a puzzle with difficulty based on the player's profile. • The player attempts to solve the puzzle through various interactions. • The system provides feedback on the player's input, whether correct or incorrect. • The player either solves the puzzle successfully or triggers consequences (e.g., alerting the agent or losing progress). • Solving the puzzle opens new areas, unlocks items, or progresses the story.
Postconditions	• The player has solved the puzzle and moves to the next area. • Unsuccessful attempts may result in penalties or player can try again.

Table 2.2: Detailed Use-case 2

Use Case Name	USC3: Collect Items
Level	User Goal
Primary Actor	Player.
Stakeholders	Game System
Preconditions	• The player has entered the map and is actively exploring. • Items are available in the environment for collection.
Main Success Scenario	• The player discovers an item while exploring the procedurally generated environment. • The player interacts with the item. • The system checks the player's inventory for available capacity (based on weight or item slots). • If the player has capacity, the item is added to their inventory. • The player can view collected items in their inventory and use or examine them as needed.
Postconditions	• The player has successfully collected the item and stored it in the inventory. • The item can be used later for puzzle solving, crafting, or defense.

Table 2.3: Detailed Use-case 3

2.2 Functional Requirements

2.2.1 Diverse Map Variations

Following are the requirements:

1. The game shall generate a variety of maps and environments on each run.

2. The game shall not generate the same maps in consecutive runs.

2.2.2 Procedural Map Generation

Following are the requirements:

1. The system will place objects and puzzles in a way that they fit naturally into the environment and do not disrupt the visual or gameplay experience.

2. The procedural generation algorithm must create maps with a logical structure that balances randomness with gameplay requirements, ensuring that each map is solvable.

2.2.3 Puzzle Generation

Following are the requirements:

1. The system will ensure the locking of doors is in a sequence.

2. The items to unlock them are places in the 'at the time' explored rooms to ensure that the puzzles are solvable.

2.2.4 Narrative Creation

Following are the requirements:

1. The narrative must be self explanatory in terms of telling the players about the characters and the over all story.

2.3 Non-Functional Requirements

2.3.1 Reliability

1. **Error Handling:** The system should gracefully handle errors and exceptions, providing informative messages or logs to the user without crashing.

2. **Save and Recovery:** The game should automatically save player progress at regular intervals or upon level completion to prevent data loss. In the event of a crash, players should be able to resume from the last saved point.

2.3.2 Usability

1. **User Interface:** The game's user interface should be easy to navigate, allowing players to access settings, start the game, and manage their progress without confusion.

2. **Intuitive UI:** The game should support basic accessibility options and simple control schemes to accommodate the player's diverse needs.

2.3.3 Portability

1. **Single-Platform Focus:** The game should be optimized for the target platform but should be designed with considerations for future portability if necessary, such as modular code and clear architecture.

2.3.4 Difficulty Scaling

1. **Dynamic Scaling:** The game should efficiently handle increasing complexity in procedural generation and level difficulty without significant performance degradation.

2. **Adaptive Load Balancing:** The system should automatically adjust server resources based on player load to ensure consistent performance.

2.4 Domain Model

Following is the domain model of our game project.

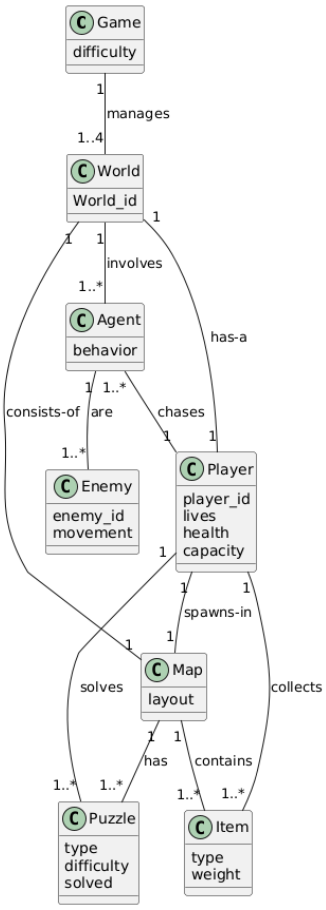

Figure 2.2: Domain Model

Chapter 3

System Overview

3.1 Architectural Design

The diagram that best fits our system is the model, view, controller (MVC) architecture, since in our system model represents the game's data, business logic and procedural generation of maps, puzzles and game elements, view handles the graphical and UI aspects of the game while controller controls game flow, manages user input and interactions.

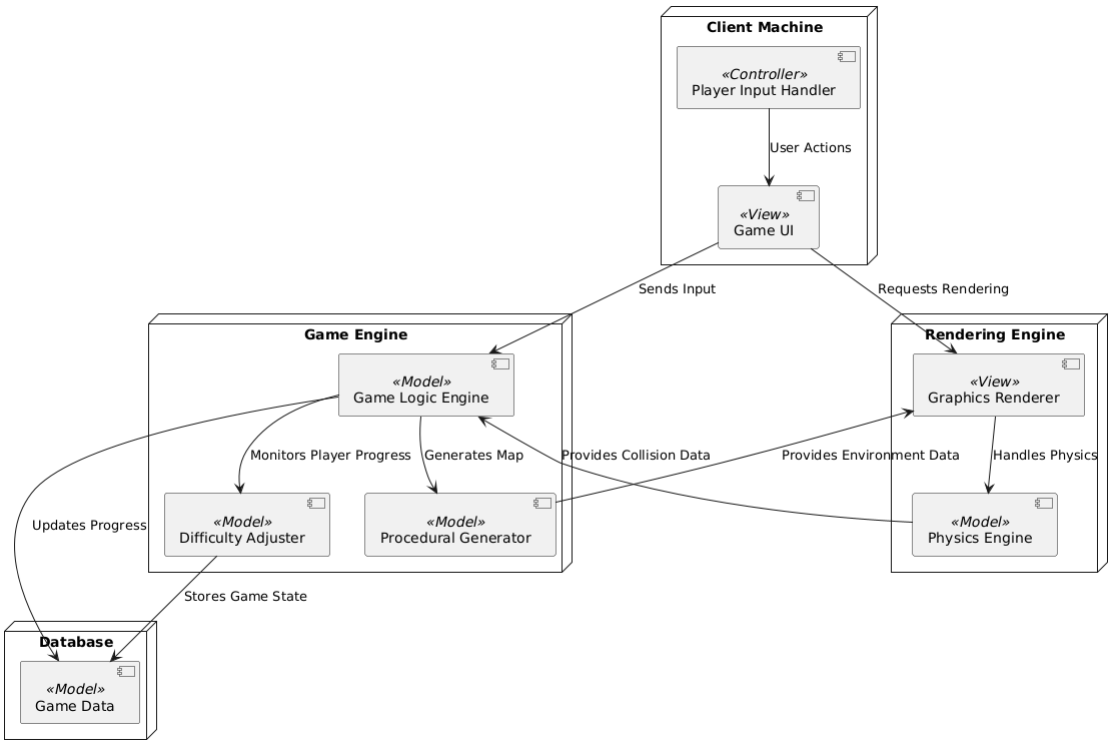

Figure 3.1: Architectural Diagram

3.2 Design Models

3.2.1 Activity Diagrams

Procedural Map Generation:

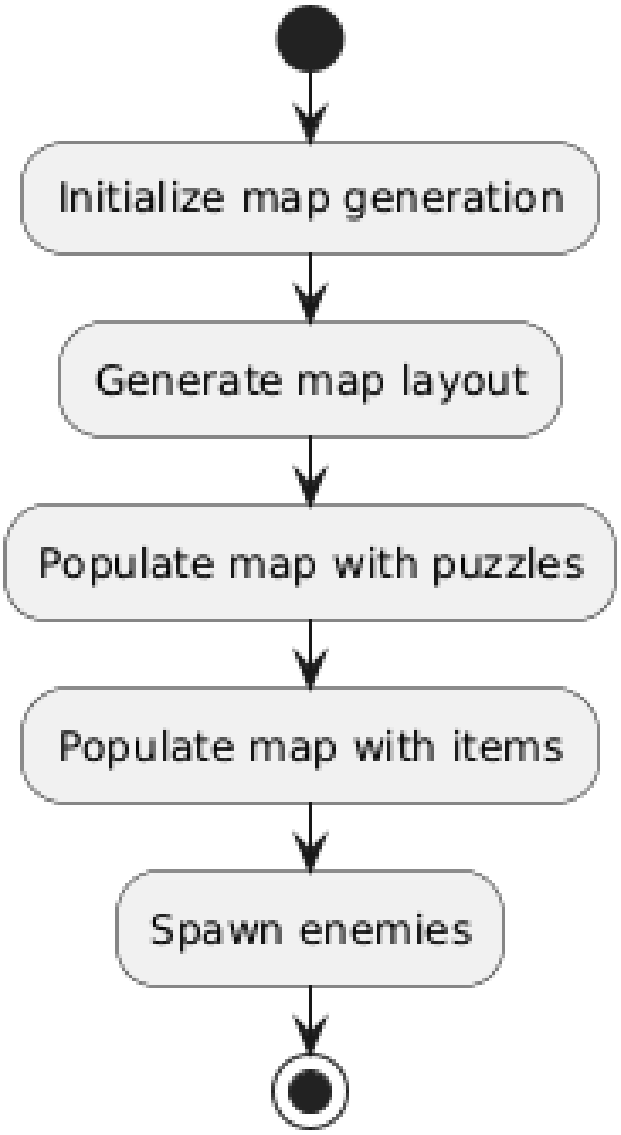

Figure 3.2: Activity Diagram 1

Exploring World:

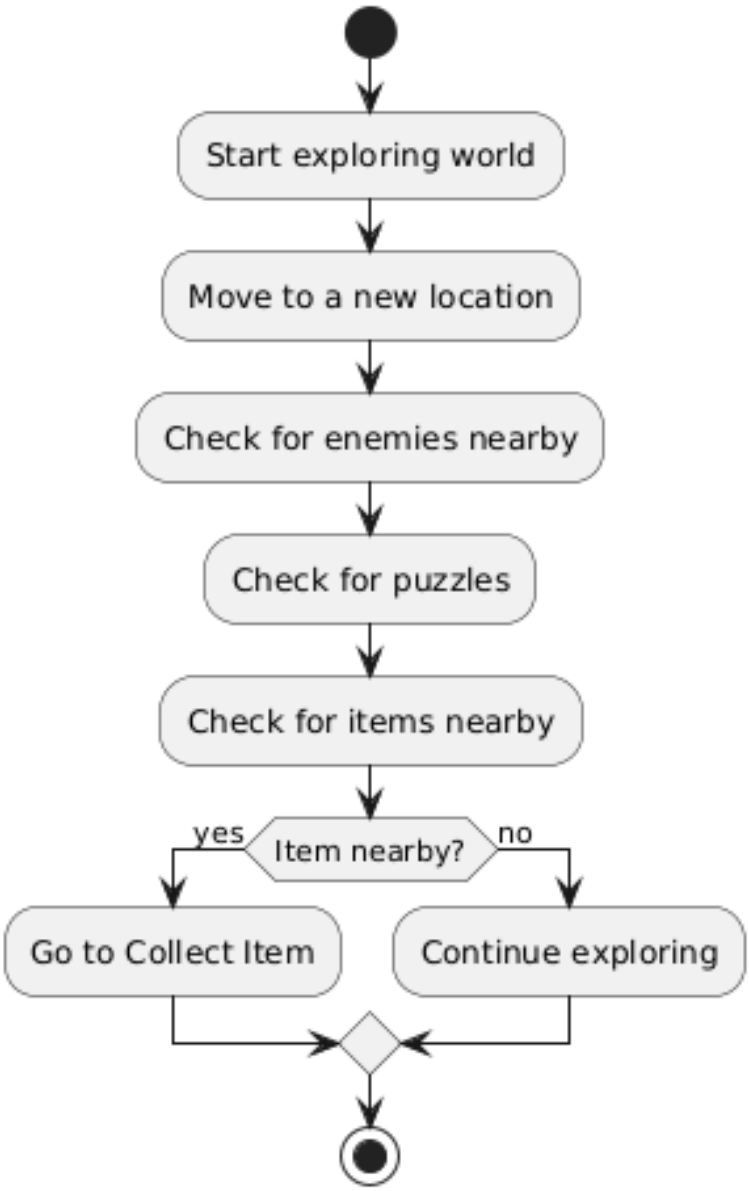

Figure 3.3: Activity Diagram 2

Collect Items:

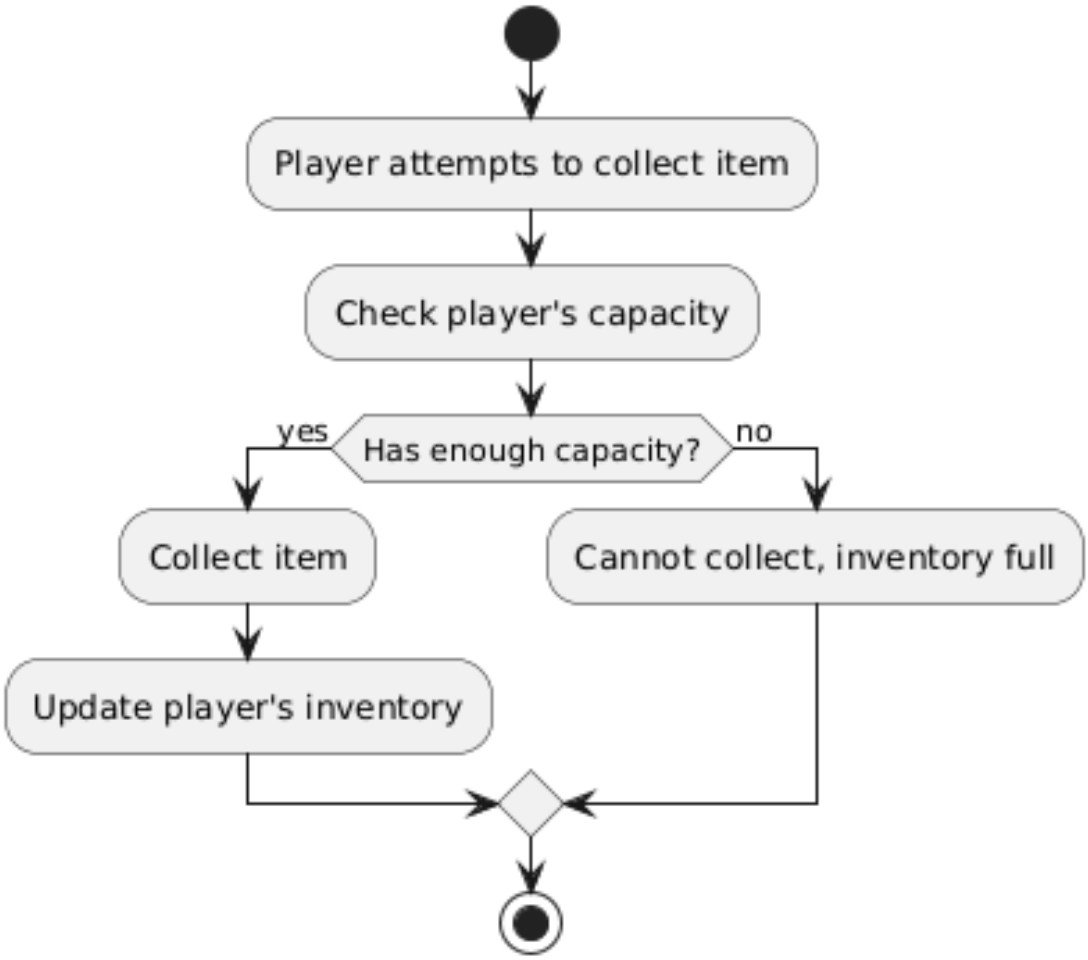

Figure 3.4: Activity Diagram 3

3.2.2 Class Diagram

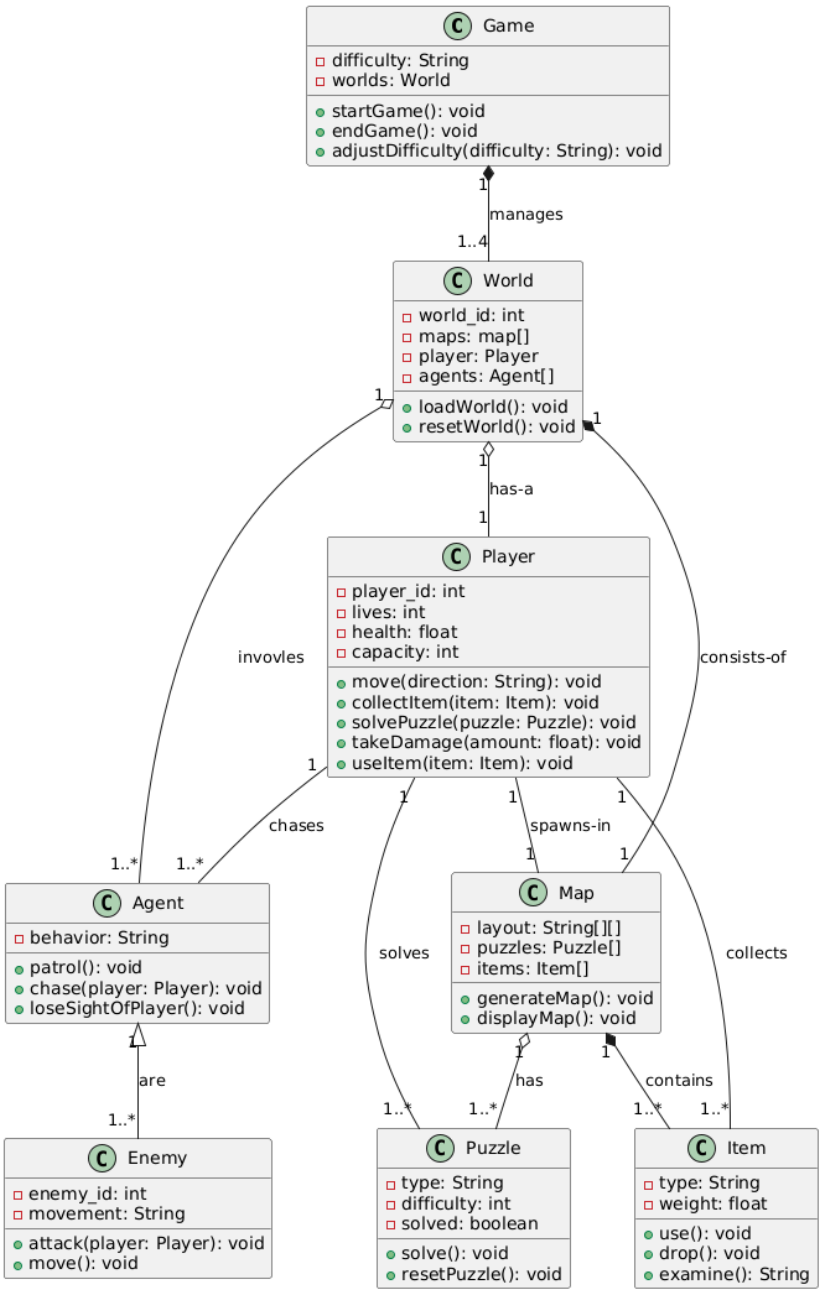

Figure 3.5: Class Diagram

3.2.3 Class-level Sequence Diagram

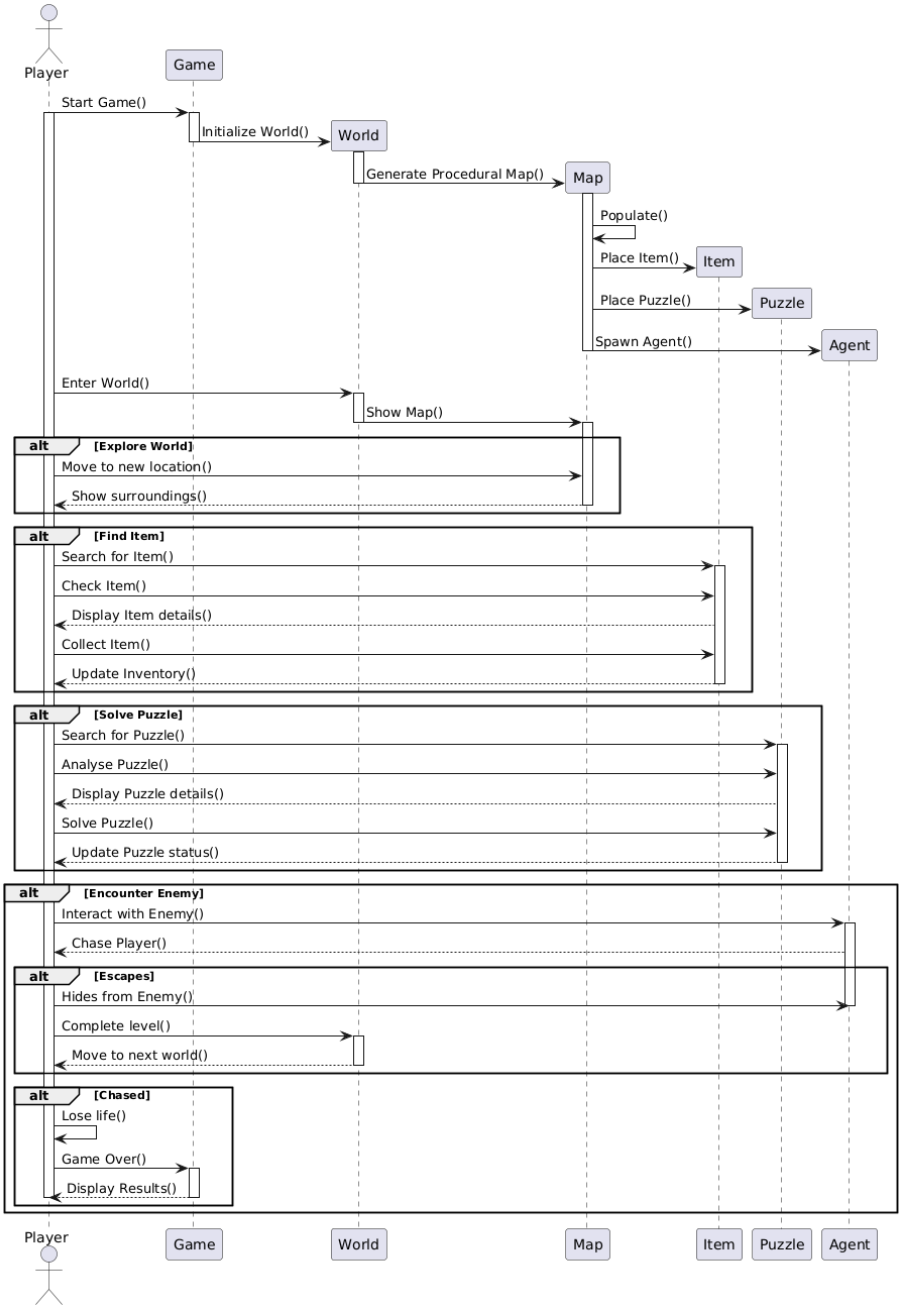

Figure 3.6: Class-Level Sequence Diagram

3.2.4 System-level Sequence Diagram

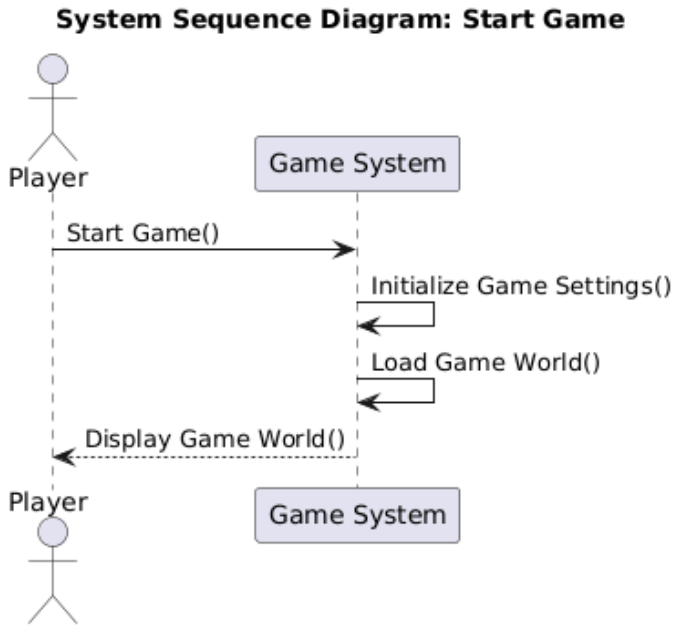

Figure 3.7: System Sequence Diagram 1

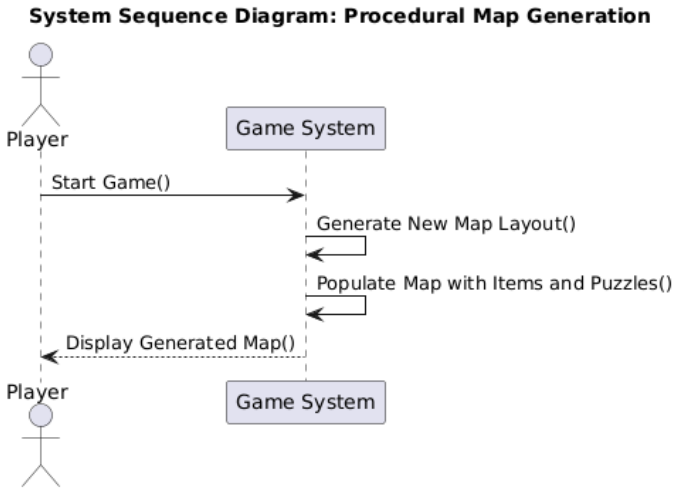

Figure 3.8: System Sequence Diagram 2

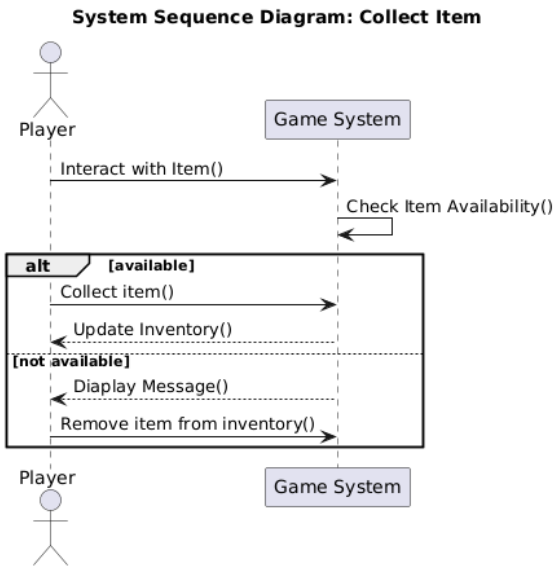

Figure 3.9: System Sequence Diagram 3

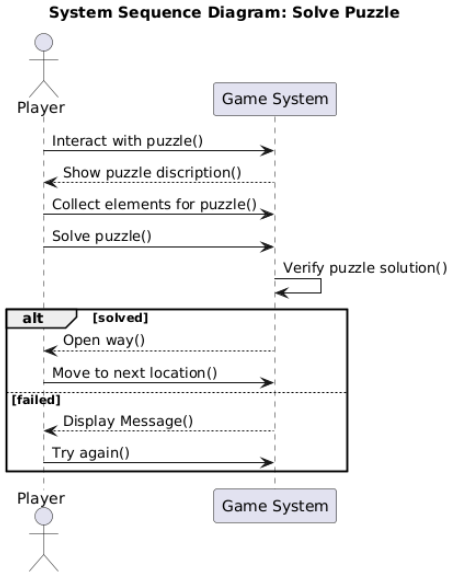

Figure 3.10: System Sequence Diagram 4

3.3 Data Design

The progress of the player will be store by using unity's file handling. In that file players and enemies coordinates and states will be stored along with the current state of the map and the seed number of the session, ensuring the progress is fully saved.

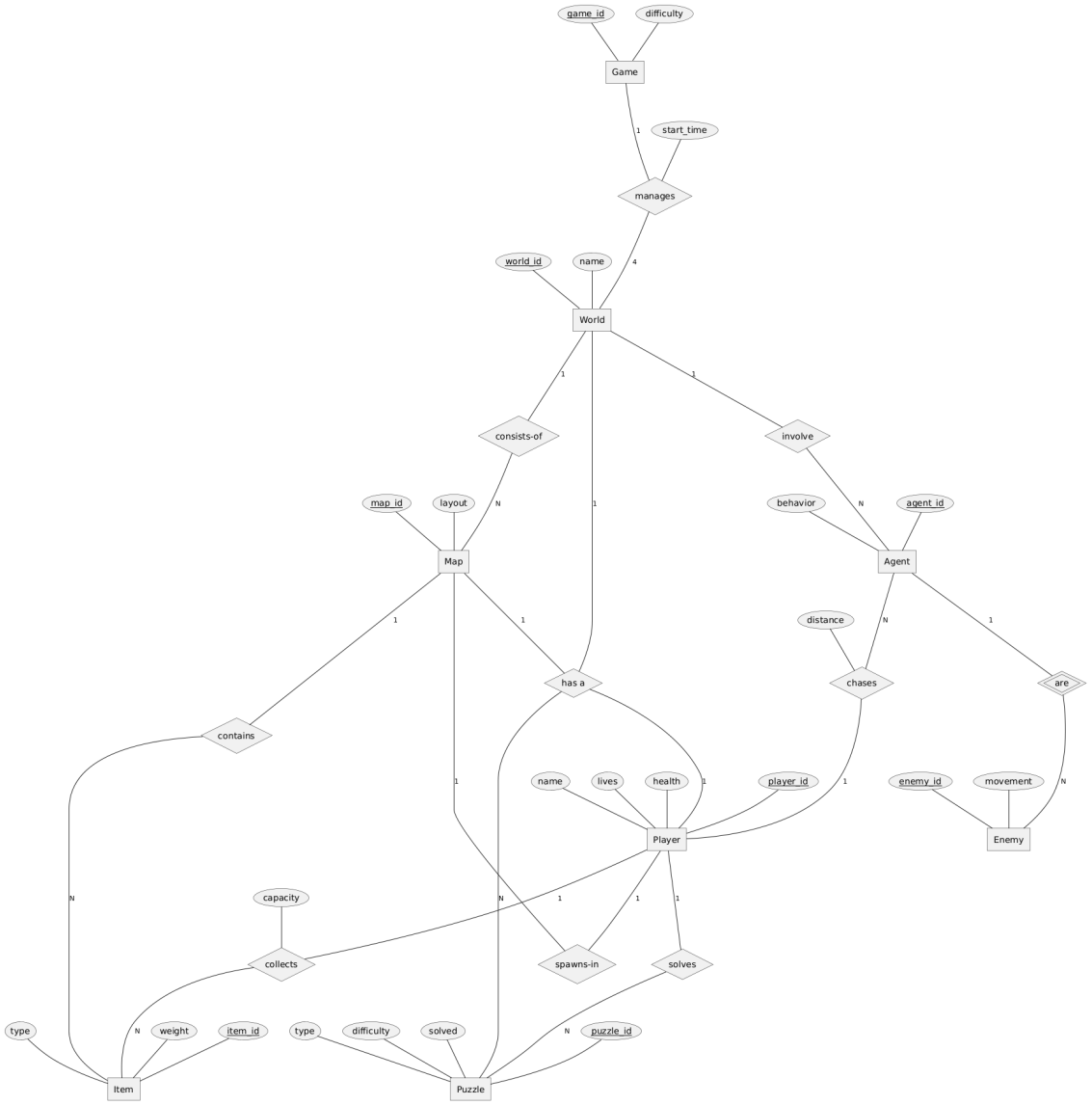

Figure 3.11: ERD

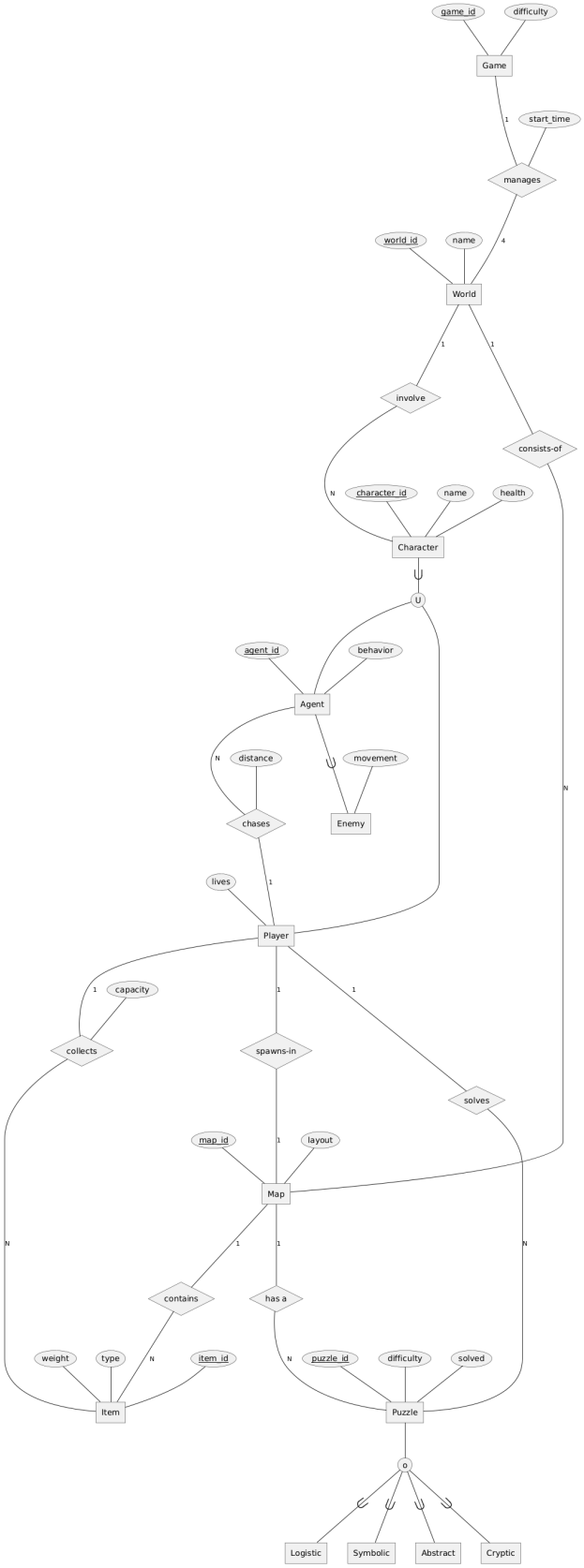

Figure 3.12: EERD

Chapter 4

Work done till current Iteration

4.1 Procedural Generation

By the current iteration we have procedurally generated some rooms of the house map. and added some very basic game mechanics like free movement.

4.1.1 implementation

We take the room dimensions as input, and on the basis of these dimensions we calculate how many blocks (general term for wall, window, doors etc) we can put in the room. For each block we make its transformation matrix and add this matrix to the list, then we use the "Graphics.DrawMeshInstanced" function, which takes mesh, transformation matrix and materials used as arguments and draws these meshes at once.

For pillars, since there are only four pillars in room, so we calculate there positions only base on room dimensions and place them using the same function above and for floor we are placing tiles row by row to make a grid that fits the whole room. For the dynamic placements of objects in room, first, we'll make a grid in room, assign each cell a tag and then place objects based on their chances of being placed in respective cells.

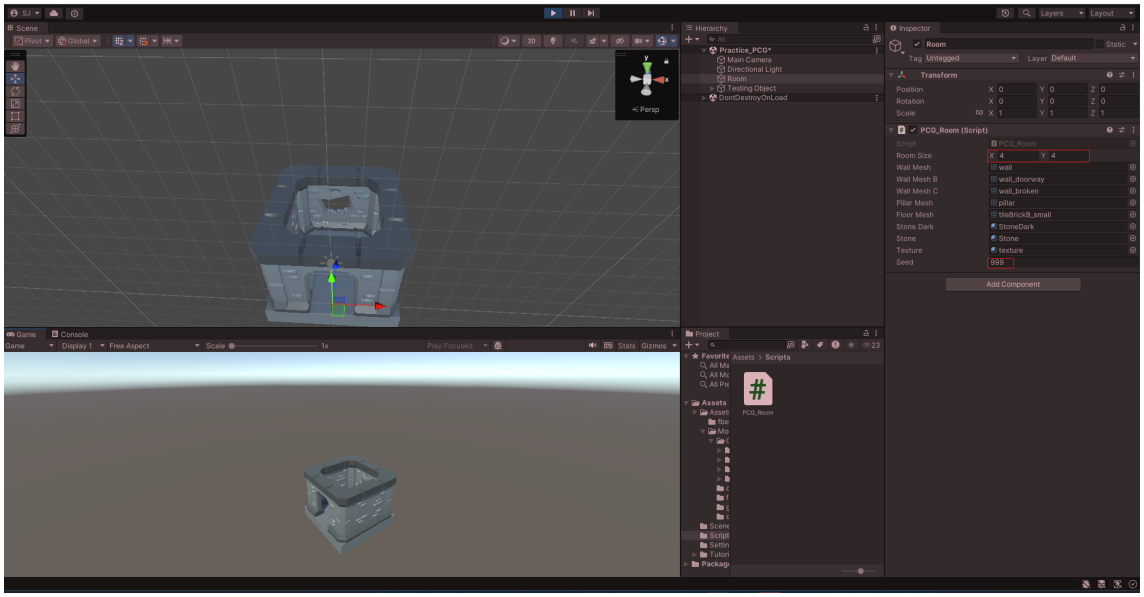

Figure 4.1: Procedurally generated room sample 1

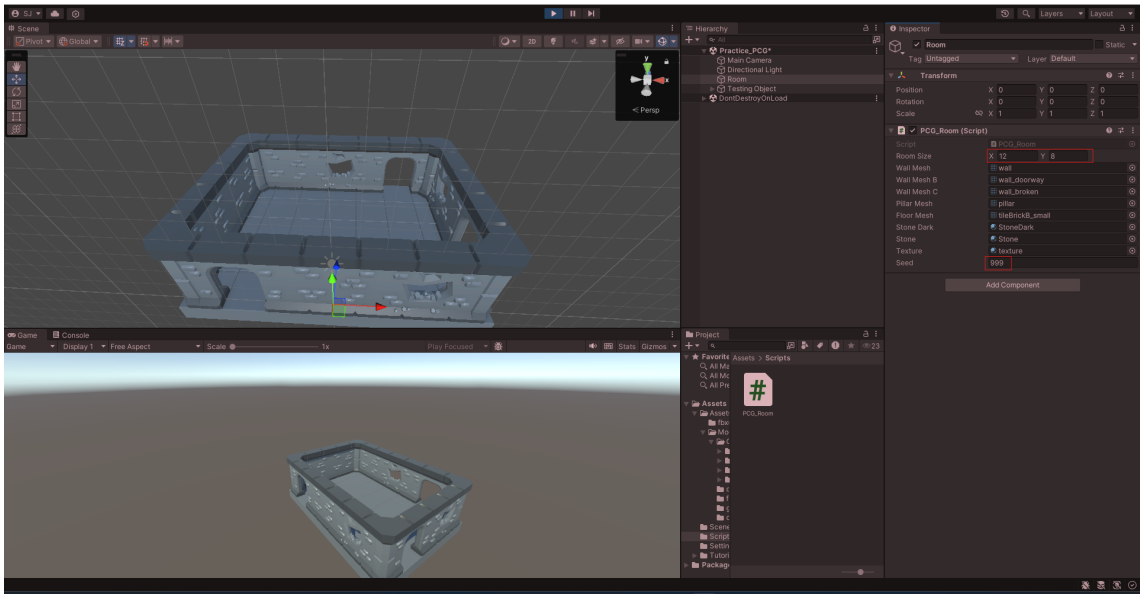

Figure 4.2: Procedurally generated room sample 2

Bibliography

[1] Jirka LP. Youtube, 2018. Accessed: 29 August 2024.

[2] M. Gamer. Youtube, 2023. Accessed: 29 August 2024.

[3] AAvirusAA. Youtube, 2022. Accessed: 29 August 2024.

[4] MoldStud. The psychology behind game difficulty: Balancing challenge and frustration, 2024. Accessed: 30 August 2024.